



Tripwire

Real-time, explainable credit-card fraud detection.

A machine-learning model scores card transactions as they arrive; a monitoring interface surfaces the suspicious ones and explains, in plain terms, why each was flagged. One coherent project sitting deliberately where financial technology, applied machine learning, and security engineering overlap — with none of the three left decorative.

DOMAIN	MODEL	SERVING	STATUS
Fraud detection	Gradient-boosted trees	Modal · scale-to-zero	Draft — v0.1

01 Synopsis

Tripwire is a real-time fraud-detection system. A model scores credit-card transactions the moment they arrive, and a monitoring interface raises the ones it considers fraudulent — together with a clear account of *why*. It is built as a single portfolio project positioned at the intersection of three competencies a hiring team reads as distinct: financial technology, applied machine learning, and security engineering.

Fraud detection is the rare problem where all three are load-bearing at once. The data is financial; the detection is a classification task under extreme class imbalance; and the function itself is a security control operating over sensitive records. Nothing here is bolted on to tick a box — and that is the point. The working thesis of this build is that depth in one well-chosen problem reads as stronger evidence than breadth across several shallow ones, and that a project one can defend in detail under questioning beats a longer list of projects one cannot.

02 Objectives & Non-Goals

Objectives

- Train and evaluate a fraud classifier on realistic transaction data, treating severe class imbalance honestly — PR-AUC and thresholded precision/recall rather than accuracy.
- Serve the model behind a scale-to-zero inference endpoint, mirroring a real ML-ops deployment pattern.
- Build a monitoring interface that streams transactions, flags predicted fraud, and — critically — explains each decision through the top contributing features and the transaction's deviation from the cardholder's baseline.
- Apply genuine security engineering to the data and request path: input validation, rate limiting, secrets handling, authentication, and a documented threat model.
- Produce the portfolio surface — README, model card, and project page — in the established editorial documentation series.

Non-Goals

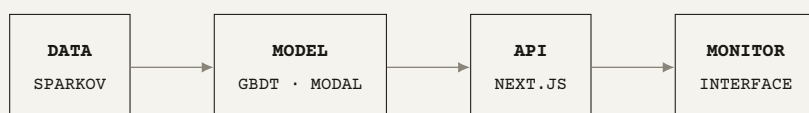
- × Not a payments system, ledger, or anything that moves money. It scores; it never transacts.
- × No real personal or financial data. The dataset is synthetic and openly licensed; no PII is ever handled.
- × Not a streaming-infrastructure project. “Real-time” here means a responsive replay of held-out transactions — enough to demonstrate the product and the model without the operational weight of Kafka or Flink.
- × Not deep learning for its own sake. Tabular fraud is a solved-shape problem for gradient-boosted trees; a neural network would be a worse, slower, less explainable choice made

only to look impressive.

- × No multi-model leaderboard, AutoML, or hyper-parameter theatre. One well-understood, well-tuned model with an honest evaluation.
- × Not a SOC, SIEM, or alerting platform.

03 System Architecture

The system is a short, legible pipeline. Transaction data is cleaned and split offline; the model trains on one split and is exported with its preprocessing and an explainer; a scale-to-zero end-point serves predictions; a Next.js application mediates all access and drives the interface. The browser never touches the model or the database directly — every call passes through the application's own server boundary, which is also where the security controls live.



PERSISTENCE & AUTH — SUPABASE · RATE LIMITING — UPSTASH · VALIDATION — ZOD

Components

- **Data layer.** Cleaned Sparkov transactions. The training split is used offline; the held-out test split is seeded into Supabase to serve as the replay source for the live feed.
- **Model.** Gradient-boosted trees trained in Colab, tracked in Weights & Biases, exported alongside its preprocessing pipeline and a SHAP explainer.
- **Inference.** A Modal endpoint, scale-to-zero: it accepts one transaction and returns a probability, a decision at the operating threshold, and the ranked feature contributions behind it.
- **Application.** Next.js (App Router) on Vercel. Route handlers proxy the replay feed and the scoring endpoint; Supabase provides Postgres and Auth; Upstash Redis enforces rate limits; Zod validates every payload.
- **Interface.** The Nothing-inspired monitor described in §07.

04 Data

The build uses the *Credit Card Transactions Fraud Detection* dataset generated with the open-source Sparkov simulator.¹ It is a simulated record of roughly 1.85 million transactions across 1,000 cardholders and 800 merchants over 2019–2020, and crucially its fields are human-readable: timestamp, merchant, category, amount, the geolocations of cardholder and merchant, and ba-

¹Kaggle, dataset [kartik2112/fraud-detection](https://www.kaggle.com/kartik2112/fraud-detection), generated with Brandon Harris's Sparkov Data Generation tool and released under a CC0 public-domain licence.

sic demographics. Fraud is well under one percent of all rows.

Why this dataset

The obvious alternative — the widely used set of anonymised European card transactions — exposes only PCA-transformed features (v_1 – v_{28}) with no merchant, location, or anything legible. A model trains on it fine, but the interface becomes a wall of meaningless numbers; you cannot show *why* a transaction was flagged. Sparkov's readable fields are what make an explainable monitor possible. The CC0 licence removes any legal friction for a public portfolio piece, and because the data is synthetic, no personal or financial information is ever handled — which keeps the project aligned with its own non-goals.

The ``live'' feed

The model trains and validates offline on the training split. The held-out test split is then seeded into the database and replayed through the interface on a timer, so the monitor scores transactions the model has genuinely never seen. This is the honest alternative to authoring a bespoke synthetic generator: it sidesteps the circular “you designed the very fraud you then detect” critique while keeping a single, simple data pipeline.

Known traps

Three details matter, and being able to discuss them is itself the signal:

- **Label leakage via merchant names.** A large share of fraudulent rows carry merchant strings that literally contain the token “fraud”. Left in, the model learns the substring and reports near-perfect, meaningless scores. The pipeline strips this before training.
- **Extreme imbalance.** At well under one percent positives, accuracy is worthless — a model that always predicts “not fraud” scores over ninety-nine percent. Evaluation therefore rests on PR-AUC and on precision/recall at a chosen threshold, with class weighting applied during training via `scale_pos_weight`.
- **Geography.** Raw latitude/longitude of cardholder and merchant are weak on their own; the useful signal is the cardholder-to-merchant distance and how far it deviates from that card's norm — a feature-engineering point, not a raw column.

Feature engineering

Derived, causal features include: transaction hour and day-of-week; amount expressed as a z-score against the cardholder's history in that category; haversine distance between cardholder and merchant; time since the card's previous transaction and a rolling transaction velocity; and category encodings. Rolling and historical features are computed strictly in time order so that no future information leaks into a row's own prediction.

05 Model

Choice of model

Gradient-boosted decision trees (XGBoost or LightGBM). The rationale is deliberate and worth stating: GBDTs dominate tabular problems, train in minutes at this volume, rank feature importance natively, pair cleanly with SHAP, and serve cheaply. A deep network would be a weaker choice dressed up as a stronger one. This is recorded as a decision (§10), not an accident.

Imbalance and threshold

Class weighting handles the skew during training. The decision threshold is then read off the precision-recall curve to hit a deliberate operating point — typically maximising recall subject to a precision floor, because a fraud team wants to catch fraud without burying analysts in false positives. That threshold is a *product* choice, not a default of 0.5, and it is surfaced in both the interface and the model card.

Metrics

PR-AUC is the headline. Alongside it: precision, recall, and F1 at the chosen threshold; a confusion matrix; and a note on calibration, since scores shown to a user should be roughly interpretable as probabilities (isotonic calibration is a stretch goal).

Explainability

Per-prediction SHAP values give the ranked feature contributions for each flagged transaction. These are not a side artefact — they are the literal content of the interface's "why flagged" panel, and they are the project's clearest ML-maturity signal.

Serving

The model, its preprocessing, and the explainer are exported behind a Modal endpoint that scales to zero, so idle cost is nil — the same pattern reused from the job-listing parser. The endpoint takes a single transaction and returns its probability, the decision at the threshold, and the ranked contributions.

06 Application & API

Framework

Next.js (App Router) on Vercel. Server route handlers mediate all access to the model and the data; the browser only ever calls the application's own endpoints.

Endpoints

- **Replay / stream** — emits held-out transactions on a timer (server-sent events or polling), each enriched with the model's score, to drive the live monitor.
- **Score** — proxies the Modal endpoint to score any single transaction, behind validation and a rate limit.

- **Aggregates** — read endpoints serving the figures behind the analytics view.

Persistence, auth, and limits

Supabase provides Postgres and Auth: the seeded test transactions, the model’s predictions, and any analyst annotations live here, with row-level security on user-scoped tables and authentication gating the dashboard. Zod validates every payload at the server boundary, and Upstash Redis rate-limits the scoring endpoint so that abuse — and runaway inference cost — is structurally impossible.

07 Interface

Design philosophy

The interface takes its language from Nothing, the consumer-tech company. Their design rests on a few principles: *expose the internals* rather than hide them; a dot-matrix, LED-like visual vocabulary; monochrome discipline broken by a single signal colour; utilitarian, grid-true precision; and restraint as a form of confidence. These usually read as hardware-industrial traits — but they map almost exactly onto what a fraud monitor must do. A fraud monitor’s whole job is to pull signal from noise and then *justify* its calls. So “expose the internals” becomes the explainability panel, and the single signal colour becomes the meaning of a flag. The aesthetic and the function turn out to be the same idea, which is why the design is a choice and not a costume. *Inspiration only — no Nothing marks, proprietary fonts, or implied affiliation; license-clean analogues throughout.*

Design tokens

TOKEN	SPECIFICATION
COLOUR	Near-black base; a single raised surface; hairline borders; near-white primary text; grey secondary text. One accent — signal red (#D8232A)  , reserved exclusively for fraud / alert states. No green: cleared transactions read as neutral, never “good”, keeping the palette monochrome-plus-one.
TYPE	Display & numerics: a dot-matrix face (Doto , openly licensed, as a clean stand-in for Nothing’s proprietary dotted type). UI text: a neutral grotesque (Inter, carried from the wider portfolio). Data: a monospace (Space Mono / Geist Mono). Figures are tabular-aligned everywhere they matter.
GRID	A strict modular grid; bento-style cards with hairline borders and generous negative space; a faint dot-grid background as a nod to dot-matrix displays; an 8-px baseline.
MOTION	Minimal and mechanical: a dot-matrix “boot” loading state, transactions arriving on a precise tick, and one live indicator that pulses as the feed runs and flashes red on a flag — a software echo of the Glyph.

Key screens

- **Live Monitor** (the hero). A streaming column of transactions, each row showing merchant, amount, and a status dot; a large dot-matrix readout of the current risk score and running counters (transactions per minute, flags caught); and the pulsing live indicator. Flagged rows carry the signal red; everything else stays monochrome.
 - **Transaction Detail** — the “exposed internals”. Opening a flagged transaction lays the model’s reasoning bare: the score against the threshold, the top SHAP contributions in plain language, and the transaction’s deviation from the cardholder’s baseline (amount against their category norm, distance from usual locations, odd hour, velocity). This is the transparency principle made literal and the interpretability story made visible.
 - **Analytics** — fraud rate over time, precision and recall at the live threshold, a confusion matrix, and category breakdowns, all drawn in the same monochrome-plus-red, dot-matrix-labelled language.
 - **Model Card** — one page stating the training data, the metrics, the chosen threshold and its rationale, and honest limitations. Cheap to build, and a strong signal of ML maturity.
-

08 Security

What makes this a security project, concretely, is a documented threat model and the controls that answer it — applied with intent rather than by reflex.

Threat model

The assets are the model endpoint and its cost, the data, and the authentication boundary. The plausible abuses are endpoint abuse and cost exhaustion, injection through unvalidated input, scraping, and auth bypass. Each maps to a control below.

Controls

- **Input validation.** Zod schemas at every server boundary reject malformed or out-of-range transaction payloads before they reach the model.
- **Rate limiting.** Upstash Redis on the scoring endpoint bounds abuse and makes runaway inference cost impossible.
- **Secrets.** The Modal token and Supabase service keys stay server-side only; the browser talks solely to the application’s own route handlers.
- **Authentication.** Supabase Auth gates the dashboard, with row-level security on user-scoped tables.
- **Transport & headers.** HTTPS throughout and a security-headers pass (CSP, HSTS, X-Frame-Options, and the rest) — the same hardening done on the job tracker, applied here deliberately.

Two levels of security

Fraud detection is itself a security function — the canonical example of anomaly detection as a defensive control. The project therefore demonstrates security thinking at two levels at once: the application's own posture, and the thing the application does.

09 Build Plan

This is a larger build than a single weekend — a deliberate choice in service of standing out. The phasing and honest estimates are below; an MVP cut follows for when time is tight.

PHASE	WORK	EST.
P0	Spec & scaffold: this document; repo, Next.js + Tailwind + shadcn scaffold, design tokens, Supabase project, Modal account.	3h
P1	Data & model: acquire and clean Sparkov (strip the merchant-name leak), causal feature engineering, train and tune the GBDT, evaluate (PR-AUC, threshold), SHAP, export the pipeline.	6--8h
P2	Serving: Modal scale-to-zero endpoint; request/response schema; smoke tests.	3h
P3	Backend: Supabase schema and seed the test split; replay/stream endpoint; scoring proxy; Zod; Upstash rate limit; auth.	5h
P4	Interface: the Nothing-inspired design system in Tailwind; Live Monitor; Transaction Detail; Analytics; Model Card.	8--10h
P5	Hardening & portfolio: threat-model write-up, security-headers pass, secrets review; README, model card, project-page copy.	4h
Total		30--38h

MVP cut. Cleaned data, one tuned GBDT, the Modal endpoint, the Live Monitor, the Transaction Detail panel, and the security-headers / rate-limit pass. That alone exercises all three domains and demos well; Analytics, the Model Card page, and authentication become stretch goals.

10 Decisions

D-01 Sparkov over the anonymised set.

Readable features make an explainable interface possible; the PCA-anonymised set would render a wall of meaningless numbers.

D-02 Replay the held-out test split.

One honest pipeline; the model scores unseen data, which avoids the circular “design-then-detect” critique a bespoke generator invites.

D-03 Gradient-boosted trees over a neural network.

Better and faster on tabular data, natively explainable, cheap to serve. A deep net would be weaker signal pretending to be stronger.

D-04 PR-AUC and thresholded precision/recall over accuracy.

Mandatory under sub-one-percent imbalance, where accuracy rewards a model that never predicts fraud.

D-05 Nothing-inspired design over the house editorial style.

A deliberate departure for this project: the transparency ethos maps to explainability and red-as-signal maps to flagging. Form follows the problem.

D-06 Modal scale-to-zero serving.

Zero idle cost and a real ML-ops pattern, reused from the job-listing parser.

D-07 No real data, no money movement.

Keeps the project legally and ethically clean, and sharpens its scope.

1.1 Definition of Done & Portfolio Framing

Done when

- **Model:** a stated PR-AUC target met on the held-out split, the threshold documented, and SHAP explanations rendering per transaction.
- **App:** deployed on Vercel and Modal, the live monitor streaming, flagged transactions showing real explanations, with auth and rate limiting live.
- **Security:** the headers pass complete, inputs validated, secrets server-side, and the threat model written.
- **Portfolio:** README, model card, project page, and a CV line.

Sample CV line (draft)

Built Tripwire, a real-time credit-card fraud-detection system: a gradient-boosted model (PR-AUC X.XX on held-out data) served scale-to-zero on Modal, with a Next.js monitoring interface that explains each flagged transaction via SHAP, hardened with input validation, rate limiting, and a documented threat model.

Interview talking points

The depth this project buys: why accuracy is the wrong metric here and what replaces it; how the merchant-name label leak was found and removed; how the decision threshold was chosen as a product trade-off; why gradient-boosted trees beat deep learning for tabular fraud; and how the design's transparency principle and the model's explainability are, in the end, the same decision.

END OF SPECIFICATION 01 — TRIPWIRE — v0.1 DRAFT